

Product Requirements Document (PRD)

Content Operations & Project Memory Platform

Aaryajanani pilot context • MERN learning project • Hackathon preparation

PRD v1.0 • 10 August 2026

Core constraint: MongoDB is mandatory. PostgreSQL is optional only if a concrete requirement justifies it.

Build principle: the developer builds the frontend, backend, database, authentication, authorization, integrations and testing from scratch; the MVP must be genuinely functional, not a mockup.

1. Product Overview

A centralized content-operations platform for a small social-media team. The central abstraction is the Content Project: one persistent record connecting a concept to production, discussion, drafts, revisions, approval, publication and performance.

MVP scope is one organization. The schema should remain organization-aware so future multi-organization support does not require a fundamental rewrite.

2. Product Vision

Make a piece of content behave like a persistent project record rather than a collection of messages, spreadsheet rows, files and links scattered across tools.

Long-term lifecycle: concept → collaboration → production → review → revision → approval → publication → performance → learning.

3. Goals

- Create one reliable project context for each piece of content.
- Reduce manual status-entry work.
- Make current project state visible without asking teammates.
- Keep drafts, versions, feedback and approvals connected.
- Make the approved final asset discoverable.
- Connect a project to its published social post.
- Attach performance to the originating project.
- Implement real authentication and authorization.
- Use the project as serious MERN practice: APIs, database design, CRUD, files, notifications, testing, errors and deployment.
- Build a production-minded MVP rather than a superficial prototype.

4. Non-Goals for MVP

- Multi-organization UI and cross-organization workflows.
- Automatic Google Meet transcription.
- AI meeting summaries/content generation.
- Automatic Instagram/YouTube publishing.
- Advanced automated social analytics.
- Payroll.
- Real-time collaborative video editing.
- Replacing professional editing/design software.
- Rebuilding every project-management or social-scheduling platform.

5. Actors

Actor	Responsibilities
Admin / Team Lead	Manage members; assign work; manage projects; approve; schedule/publish; permissions; operational dashboards.
Content / Strategy Member	Concepts; research; scripts; footage/voiceover; inputs; reviews; publishing when permitted.
Editor / Designer	Assigned work; inputs; drafts; comments; revisions; project history.
Reviewer	Review drafts; comment; request changes; approve where authorized.
Scheduler / Publisher	Approved/scheduled projects; final assets; publication records.

6. Authorization Model

Separate roles from permissions. Conceptual model:

- User
- Organization
- Organization Membership
- Role
- Permission
- Project-level assignment/access where required

Server-side authorization is mandatory. Frontend hiding alone is not security.

7. Central Domain Object — Content Project

Area	Information
Identity	Project ID, title, type, description, status, dates.
Concept	Original concept, proposer, discussion/decision.
People	Creator, assignee, reviewers, publisher.
Inputs	Required inputs, owner/source, received state, asset references.
Production	Drafts, versions, files, timestamps.

Review	Comments, reviewers, decisions, requested changes.
Revision	Revision number, reason, requested changes, submission.
Approval	Exact approved version, approver, timestamp.
Scheduling	Planned publication date/time.
Publication	Platform, post URL/ID, published timestamp.
Performance	Metrics linked to the published project.
History	Activity/event timeline.

8. Project Types

Use a small set of explicit types inspired by the user's GitHub Issue mental model; do not clone GitHub.

Type	Purpose
New Concept / Content	Propose and develop new content.
Experiment	Track a deliberate content experiment and result.
Revision	Track a revision connected to existing work.
Content Production	Represent active production of a video/post/design.

9. Core Lifecycle

1. Concept proposed.
2. Team discusses and agrees to pursue it.
3. Concept becomes a project.
4. Project assigned.
5. Inputs identified/requested.
6. Inputs received.
7. First draft produced and uploaded.
8. Reviewers notified.
9. Review/comments happen in project context.
10. Revision request created when needed.
11. New version submitted.
12. Review repeats.
13. Authorized reviewer approves exact version.
14. Approved asset becomes final.
15. Project scheduled.
16. Publisher records social post URL/ID.
17. Performance metrics recorded in MVP.
18. Project remains available as historical context.

10. Status Model

Status	Meaning	Trigger
IDEA	Concept exists.	Concept created.
APPROVED_CONCEPT	Team agreed to pursue it.	Concept discussion concluded.
ASSIGNED	Responsible member assigned.	Assignment.
WAITING_FOR_INPUTS	Production waiting for required	Inputs requested.

	inputs.	
INPUTS_READY	Inputs available.	Inputs received.
IN_PROGRESS	Production active.	Work starts.
FIRST_DRAFT_SUBMITTED	Draft ready for review.	Draft upload.
UNDER_REVIEW	Review active.	Review begins.
REVISION_REQUESTED	Changes required.	Revision request.
REVISION_IN_PROGRESS	Changes being made.	Revision starts.
REVISION_SUBMITTED	New version ready.	Revision upload.
APPROVED	Authorized approval recorded.	Approval.
SCHEDULED	Publication planned.	Schedule created.
PUBLISHED	Publication recorded.	Post URL/ID recorded.
CLOSED	Lifecycle complete.	Project closed.

11. Functional Requirements

11.1 Authentication

- Email/password registration/login.
- Secure password hashing; never store plaintext.
- JWT-protected API.
- Protected routes reject unauthenticated requests.
- Logout/token expiry handled correctly.
- Google OAuth after basic authentication is stable.

11.2 Organization & Members

- Admin manages members and roles.
- Membership status can be enabled/disabled.
- Organization boundary enforced server-side.
- MVP supports one organization in UI.

11.3 Concepts & Projects

- Create concept.
- Convert concept into project.
- Persistent project ID.
- Assign responsible member.
- Project status changes from meaningful actions.

11.4 Inputs

- Define required inputs.
- Track input owner/source.
- Track requested/received/missing/blocked state.
- Receiving an input should update relevant project state without duplicate spreadsheet entry.

11.5 Drafts & Versioning

- Upload drafts.
- Every draft has explicit version identity.
- Previous versions remain accessible.

- Current version is obvious.
- Approved version is explicit.
- Do not overwrite historical versions.

11.6 Review & Comments

- Authorized reviewers comment on a version.
- Responsible member can reply.
- Comments retain project/version context.
- Discussion supports disagreement and negotiation.
- Reviewer can request revision.

11.7 Revisions

- Revision linked to project and source version.
- Record number, reason, requester, submitter and timestamps.
- Submission changes project state automatically.
- Revision count visible.
- Limit should warn/escalate rather than hard-block by default.

11.8 Approval

- Authorized reviewer approves exact version.
- Record approver and timestamp.
- No scheduling without approved version.
- Approval auditable.

11.9 Scheduling & Publication

- Set planned publication date/time.
- Publisher accesses approved asset.
- Record platform and post URL/ID.
- Record publication timestamp.
- MVP records publication; it does not automatically publish.

11.10 Performance

- Manually enter initial metrics.
- Metrics linked to project/publication.
- Display basic performance from project page.
- Future adapter can automate ingestion.

11.11 Notifications

- Draft → reviewers.
- Comment/revision → responsible member.
- Revision submission → reviewers.
- Approval → publisher.
- Publication/important events → relevant stakeholders.

11.12 Activity History

- Record important state changes with actor, action, timestamp and related entity/version.

- History remains viewable after completion.

12. Core Screens

Screen	Purpose
Login	Authentication.
Dashboard	Current work, blockers, review queue, upcoming publication, recent activity.
Projects	Search/filter/sort authorized projects.
Project Detail	Central project history and workflow.
Create Concept	Capture new concept.
Review Workspace	View draft; comment; request revision; approve.
My Work	Assigned projects.
Notifications	Attention queue.
Members / Organization	Admin management.
Basic Analytics	Published projects and manual metrics.

13. MongoDB Data Model

Start MongoDB-only. Do not add PostgreSQL merely because a hybrid architecture sounds more 'production'. Introduce it only after identifying a concrete workload or constraint.

Collection	Purpose
users	Identity/auth metadata.
organizations	Organization identity/settings.
memberships	User ↔ organization, role, permissions/status.
projects	Core project state and identity.
project_versions	Draft/revision artifacts and metadata.
comments	Project/version discussion.
revision_requests	Requested changes and resolution.
inputs	Dependencies and required assets.
publications	Platform, URL/ID, publish time.
performance_metrics	Metrics linked to publication/project.
notifications	Recipient, event, read state.
activity_events	Project audit/activity timeline.
meetings	Future meeting records/manual notes.

13.1 MongoDB Principles

- Index organizationId/projectId on high-volume collections.
- Index status, assignee and scheduled date for dashboard queries.
- Do not store unbounded comments/activity arrays in a project document.
- Use separate version records for historical drafts.
- Use transactions only when a real multi-document atomic operation requires them.
- Design indexes around actual query patterns.

14. API Requirements

Area	Representative endpoints
Auth	POST /auth/register, POST /auth/login, POST

	/auth/logout
User	GET /users/me
Members	GET/POST/PATCH /organizations/:id/members
Projects	GET/POST /projects, GET/PATCH /projects/:id
Assignment	POST/PATCH /projects/:id/assignment
Inputs	GET/POST/PATCH /projects/:id/inputs
Versions	POST /projects/:id/versions, GET /projects/:id/versions
Comments	GET/POST /projects/:id/comments
Revisions	POST /projects/:id/revisions, PATCH /revisions/:id
Approval	POST /projects/:id/approve
Schedule	POST/PATCH /projects/:id/schedule
Publication	POST /projects/:id/publications
Metrics	GET/POST /projects/:id/metrics
Notifications	GET/PATCH /notifications
Activity	GET /projects/:id/activity

15. File / Asset Handling

- Store file metadata in MongoDB.
- Store large media in object storage or a controlled local storage abstraction, not MongoDB itself.
- Metadata includes projectId, versionId, uploader, filename, MIME type, size and storage reference.
- Never use filename conventions as the only identity of a file.
- Final approved asset is explicit in the database.

16. Event-Driven Workflow

Use backend domain events so workflow actions are not tightly coupled.

Example: DraftUploaded → ProjectStatusChanged → ReviewersNotified → CommentAdded → EditorNotified → RevisionRequested → RevisionSubmitted → ReviewersNotified → Approved → PublisherNotified → Published → MetricsRecorded

17. Social Scheduling / Analytics Integration

The open-source project the user was trying to recall is very likely Postiz. Its current public materials describe it as an open-source social-media scheduling platform with team collaboration, comments, scheduling, analytics and a public API. Its GitHub repository also describes it as self-hostable and available under AGPL-3.0.

Integration strategy: Postiz is a candidate external capability, not the core of the MVP.

19. MVP: manually record Instagram/YouTube post URL and basic metrics.
20. Create an internal adapter boundary for external publishing/analytics providers.
21. After the core workflow is stable, evaluate/integrate Postiz.
22. Sync supported publication state and analytics back into the Content Project.
23. Keep a manual fallback because external APIs and permissions can change.

Important: the Content Project remains the source of organizational context. An external scheduler should not replace the project's history.

18. Google Meet Integration — Deferred

- MVP: record meeting title/date and manually capture decisions.
- Later: integrate Google Meet APIs for authorized meeting metadata/transcripts/artifacts.
- Imported meeting information should become durable project context.
- Meet integration must never block the core project workflow.

19. Security Requirements

- Server-side authentication and authorization.
- Organization isolation on every protected query.
- Role/permission checks in backend services/middleware.
- Secure password hashing.
- Secrets in environment/secret storage, never source code.
- Validate file type/size and handle uploads safely.
- Treat comments and text as untrusted input.
- Consider rate limiting/brute-force protection.
- Do not expose sensitive auth/audit information unnecessarily.

20. Non-Functional Requirements

Area	Requirement
Correctness	Core actions persist correctly and trigger expected state transitions.
Performance	Indexed, paginated common queries.
Reliability	Important transitions idempotent where practical.
Maintainability	Separate routes/controllers, services, data access and authorization.
Testing	Core workflow and authorization covered by automated tests.
Observability	Useful structured errors/logging without secrets.
UX	Avoid manual status updates when an action already implies state.
Accessibility	Core interactions usable by keyboard and readable.
Deployment	Environment-specific configuration separated from code.
Traceability	Approved version, publication and project remain linked.

21. MVP Acceptance Criteria

- User can log in and access a protected dashboard.
- Admin can manage members/permissions.
- User can create a concept and convert it into a project.
- Project can be assigned.
- Inputs can be tracked.
- Editor can upload first draft.
- Reviewers are notified.

- Reviewers can comment/request revision.
- Revision creates a new version without overwriting history.
- System records exact approved version and approver.
- Project cannot be scheduled without approval.
- Publisher can record post URL.
- Performance can be attached to publication.
- Full project activity history is visible.
- Users can find current project state without asking teammates.
- Unauthorized users cannot access outside their scope.
- Core workflow has automated tests.
- No core MVP button is a fake/dead-end interaction.

22. Recommended Build Order

24. Repository/environment setup.
25. MongoDB + data-access layer.
26. Express API foundation + validation + errors.
27. Authentication + JWT.
28. Organization/membership/permissions.
29. Project/concept CRUD.
30. Project detail UI.
31. Assignment/status transitions.
32. Inputs/dependencies.
33. File metadata + draft upload.
34. Versioning.
35. Comments/review.
36. Revision workflow.
37. Approval.
38. Notifications.
39. Activity timeline.
40. Scheduling/publication record.
41. Manual analytics.
42. Dashboard/search/filtering.
43. Automated tests + authorization tests.
44. Deployment/production configuration.
45. Only then: Postiz/social integrations.

23. Definition of Done

- Frontend implemented.
- Backend implemented.
- MongoDB persistence implemented.
- Validation and authorization implemented.
- Loading/error/success states handled.
- Relevant activity/notifications recorded.
- Automated tests added for important logic.

- End-to-end scenario works.
- No core UI interaction is decorative.
- Data remains traceable from concept to publication/performance.

24. Risks & Mitigation

Risk	Mitigation
Scope explosion	Freeze MVP; defer AI, Meet, social APIs and multi-tenancy.
Hybrid DB overengineering	MongoDB first; add PostgreSQL only with evidence.
Large UI / incomplete backend	Judge completeness by end-to-end workflows.
Media storage complexity	Abstract storage; MongoDB stores metadata.
Notification coupling	Use simple domain-event pattern.
Authorization bugs	Backend checks + automated tests.
External API instability	Adapter boundary + manual fallback.
Premature AI	Only after core workflow works.

25. Success Metrics for Pilot

- Time for editor to submit first draft without separate spreadsheet maintenance.
- Time for lead to determine project/team status.
- Time for publisher to locate approved final asset.
- Percentage of projects with explicit approved version.
- Percentage of published projects linked to post URL.
- Percentage with performance attached.
- Number of times team must ask group for status/file location.
- Number of duplicate manual data-entry actions per project.
- Percentage of core workflow completed without leaving platform.

26. Future Scope

- Multi-organization support and organization switching.
- Google OAuth.
- Google Meet integration.
- Postiz integration.
- Direct Instagram/YouTube integrations where justified.
- Automatic analytics ingestion.
- AI extraction of meeting decisions.
- AI performance insights.
- Experiment hypothesis/result tracking.
- Cross-project content intelligence.
- Automated reminders/dependency escalation.
- Advanced dashboards.

27. Product Thesis

A content team should not have to reconstruct a project's history from chat messages, spreadsheets, file folders and social platforms.

The MVP makes the Content Project the persistent unit of work and connects its important states and artifacts. Specialized external tools may eventually handle publishing and analytics, but the project record preserves the organizational context.

28. External Reference — Postiz

Likely project identified during research: Postiz.

Official site: <https://postiz.com/>

GitHub repository: <https://github.com/erodium/postiz>

This identification should be verified before implementation; it is currently the best match to the open-source scheduler/analytics project described in the planning discussion.